

PRACTICAL-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

FACTORIAL BY ITERATIVE

ALGORITHM:

Input : A single integer n ($n \geq 0$)
Output : factorial of the given integer n !

⇒ start
⇒ Read an integer n
⇒ set $fact = 1$
⇒ Repeat for $i = 1$ to n :
 multiply $fact = fact * i$
⇒ End loop
⇒ to print the value of $fact$ (which is $n!$)
⇒ stop

PROGRAM:

```
#include <iostream>

using namespace std;

int factorial_iterative(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    return fact;
}
```

```
int main() {  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
    cout << "Factorial (Iterative): " << factorial_iterative(n) << endl;  
    return 0;  
}
```

OUTPUT:

```
Enter a number: 5  
Factorial (Iterative): 120
```

TIME COMPLEXITY:

```
int factorial_iterative(int n)  $\rightarrow T(n)$   
{  
    int fact = 1;  $\rightarrow 1$   
    for (int i = 1; i <= n; i++)  $\rightarrow n$   
    {  
        fact = fact * i;  $\rightarrow 1$   
    }  
    return fact;  $\rightarrow 1$   
}
```

$$T(n) = n + 3$$
$$T(n) = O(n) \quad (\because \text{By asymptotic notation})$$

FACTORIAL BY RECURSIVE

ALGORITHM:

input : integer n ($n \geq 0$)
output : factorial of integer $n!$
⇒ start
⇒ Read an integer n
⇒ If $n=0$ or $n=1$,
 return 1
⇒ otherwise :
 return $n * \text{factorial}(n-1)$
⇒ print the result
⇒ stop.

PROGRAM:

```
#include <iostream>

using namespace std;

int factorial_recursive(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial_recursive(n - 1);
}

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Recursive): " << factorial_recursive(n) << endl;
    return 0;
}
```

OUTPUT:

```
Enter a number: 5
Factorial (Recursive): 120
```

TIME COMPLEXITY:

```
int factorial_recursive (int n) → T(n)
{
    if (n==0 || n==1) → 1
        return 1;
    else
        return n * factorial_recursive(n-1)
        ↳ T(n-1)
}
```

$$T(n) = T(n-1) + O(1)$$

$$= T(n-1) + 1$$

$$T(n) = T(n-1) + 1 \rightarrow \textcircled{1}$$

$n \rightarrow n-1$ put in $\textcircled{1}$

$$T(n-1) = T(n-1-1) + 1$$

$$= T(n-2) + 1 \rightarrow \textcircled{2}$$

Substitute $\textcircled{2}$ in $\textcircled{1}$

$$T(n) = T(n-2) + 1 + 1$$

$$T(n) = T(n-2) + 2 \rightarrow \textcircled{3}$$

$n \rightarrow n-2$ put in $\textcircled{3}$

$$T(n-2) = T(n-2-2) + 2$$

$$= T(n-4) + 2 \rightarrow \textcircled{4}$$

Substitute $\textcircled{4}$ in $\textcircled{3}$

$$T(n) = T(n-4) + 2 + 2$$

$$T(n) = T(n-4) + 4 \rightarrow \textcircled{5}$$

$n \rightarrow n-4$ put in $\textcircled{5}$

$$T(n-4) = T(n-4-4) + 4$$

$$= T(n-8) + 4 \rightarrow \textcircled{6}$$

Substitute $\textcircled{6}$ in $\textcircled{5}$

$$T(n) = T(n-8) + 4 + 4$$

$$T(n) = T(n-8) + 8$$

$$\therefore T(n) = T(n-k) + k$$

$$= T(n-n) + n$$

$$= T(0) + n = 1 + n$$

$$\begin{cases} T(n) = 0 \\ n-k = 0 \\ n = k / k = n \end{cases}$$

$O(n) \rightarrow \text{time complexity}$

CONCLUSION:

The factorial program demonstrates both iterative and recursive approaches. Iterative factorial runs in linear time $O(n)$ with constant space, making it efficient. Recursive factorial also has $O(n)$ time but requires $O(n)$ stack space due to repeated function calls. Thus, iteration is more memory-efficient, while recursion is conceptually elegant.